

Arquitectura Vertical

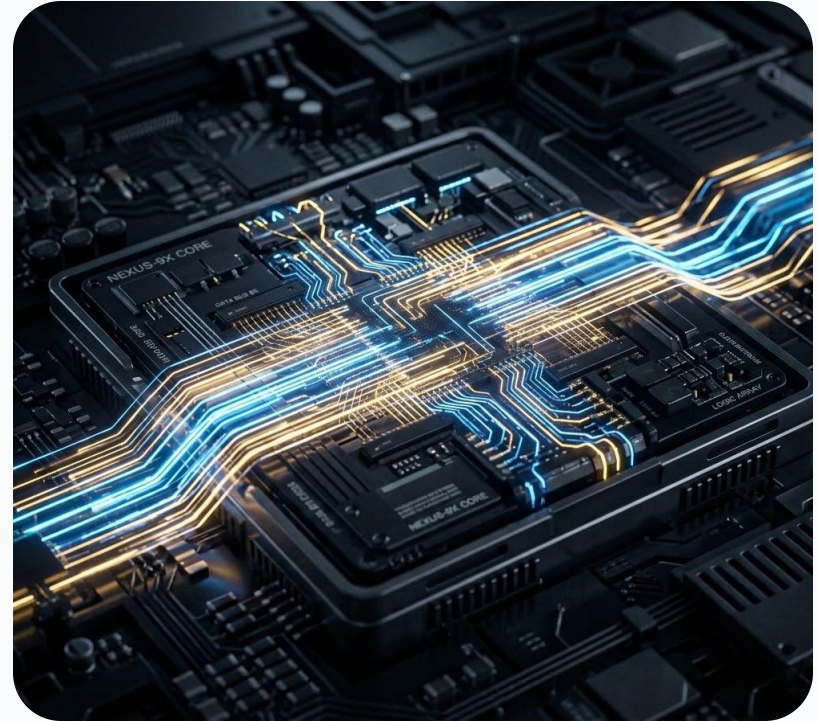
Ejecución del microprograma

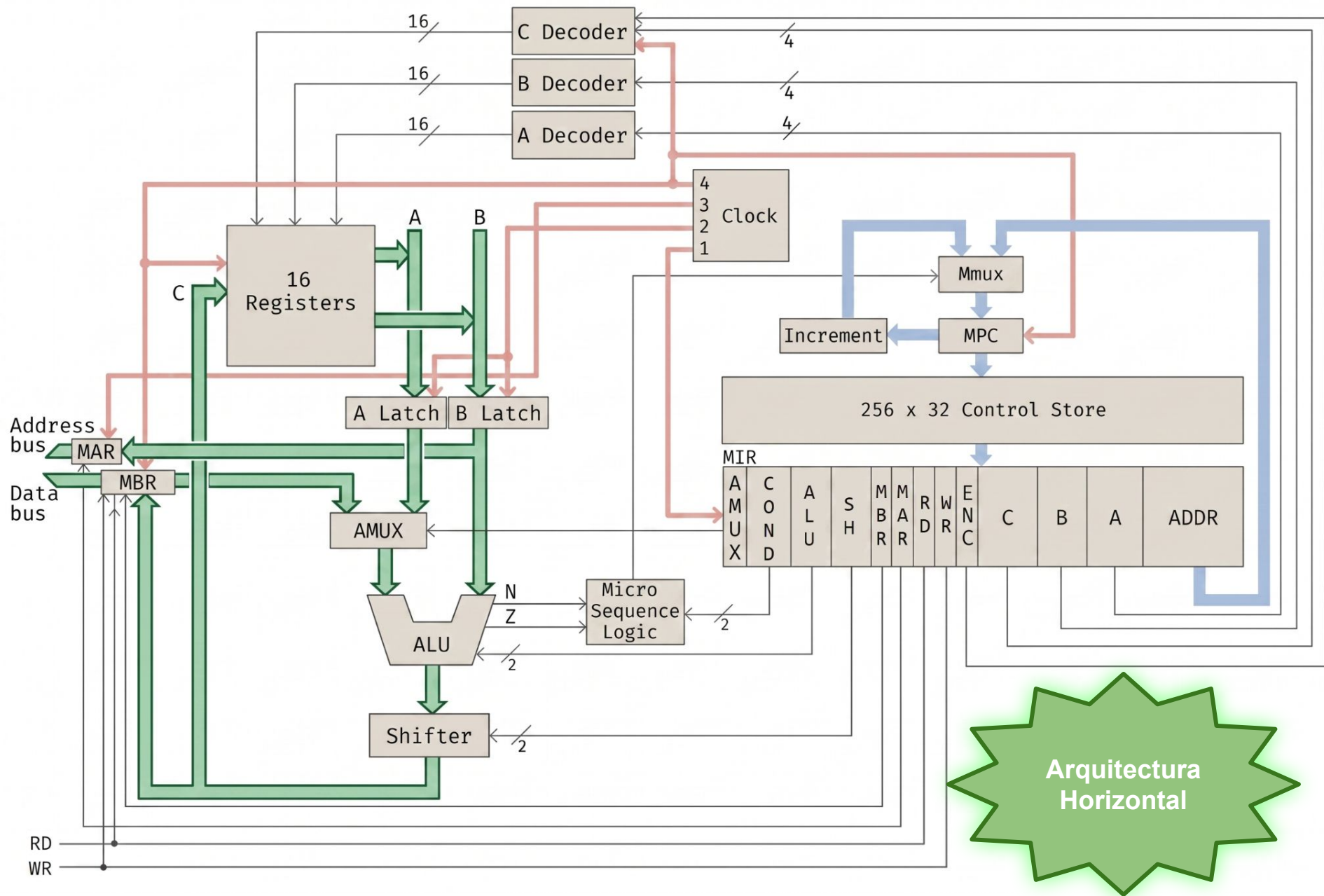


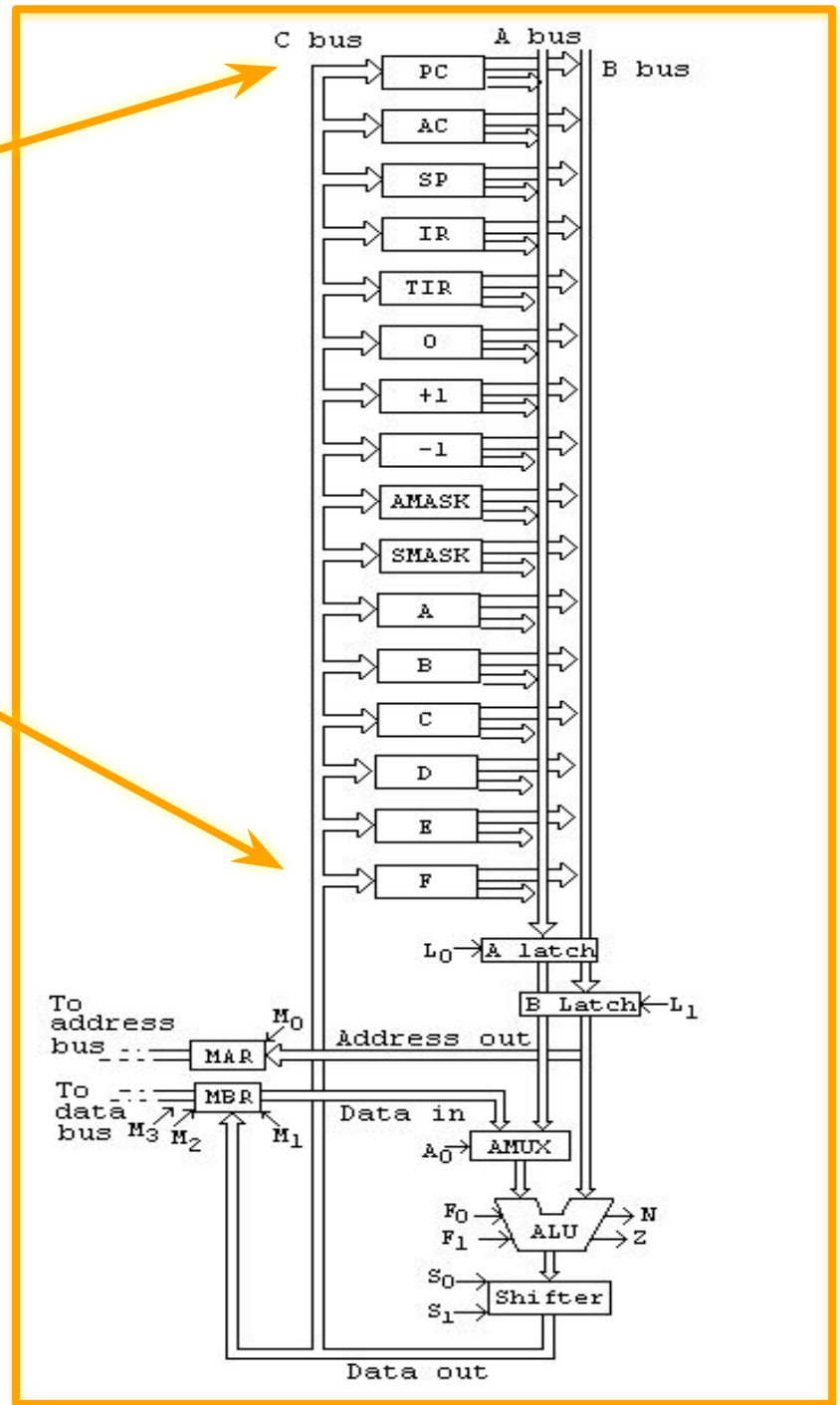
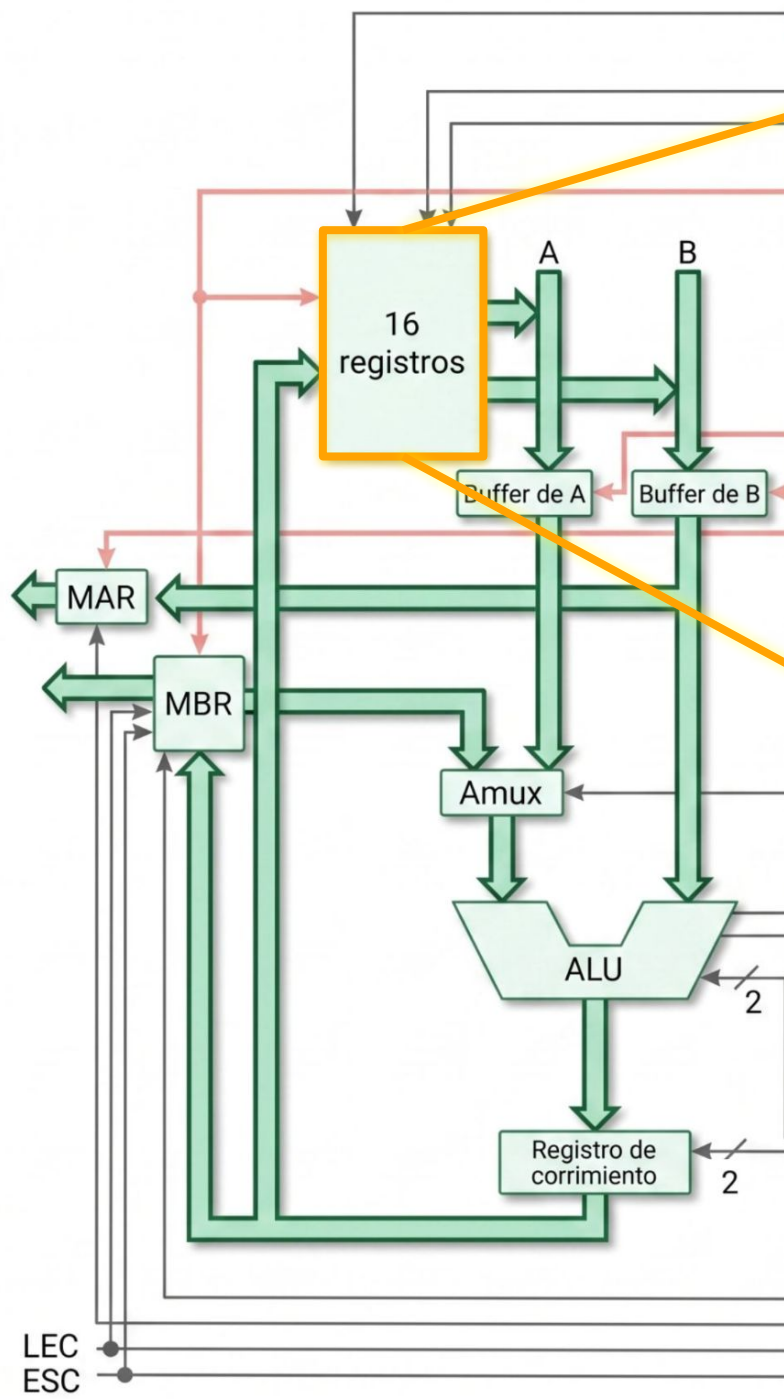
Fundamentos de Arquitectura de computadoras

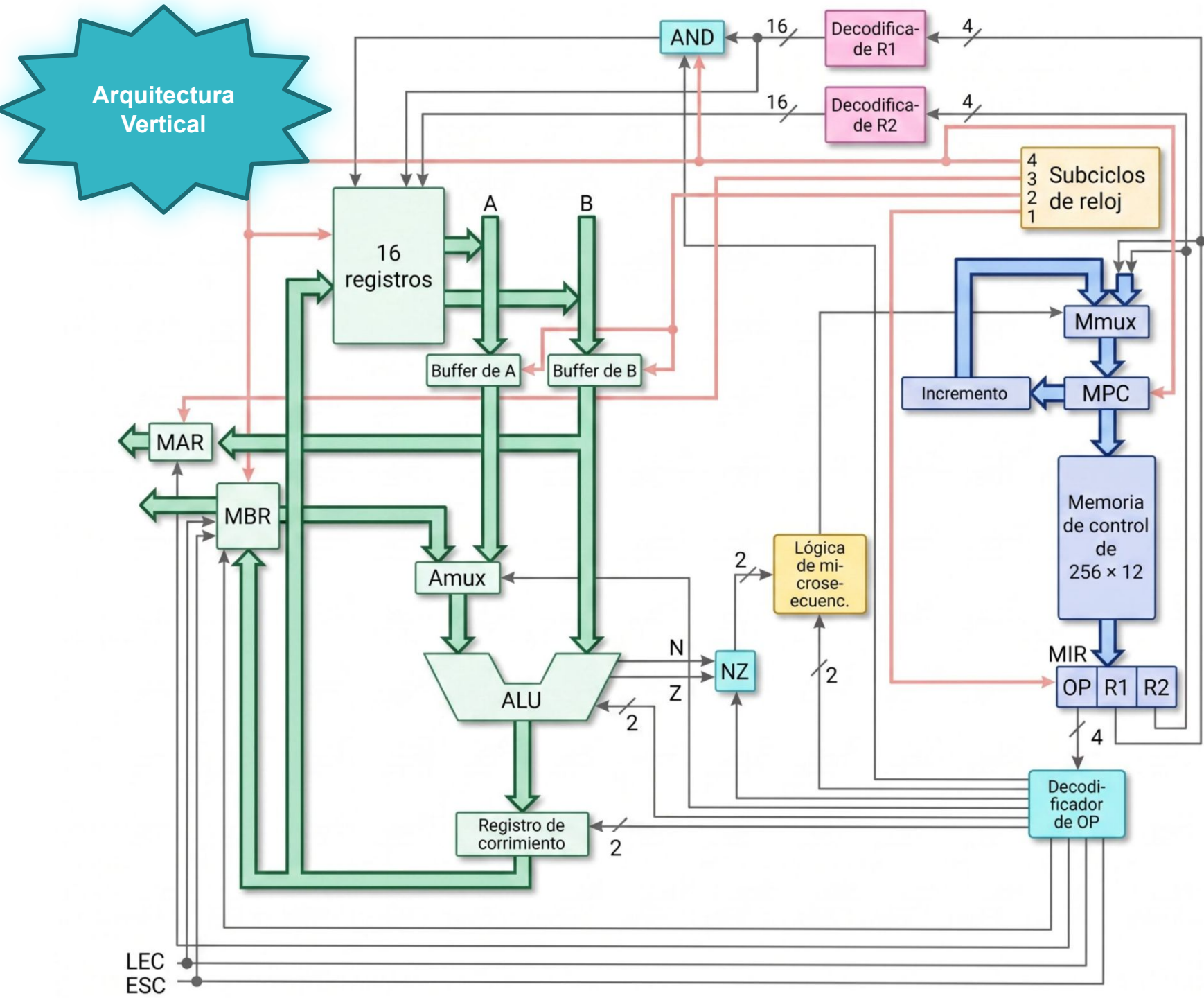
Un cambio en el control de los datos. Ahora se codifican las microinstrucciones para utilizar menos memoria de control.

- Arquitectura Vertical
- Nanoprogramción
- Pipeline

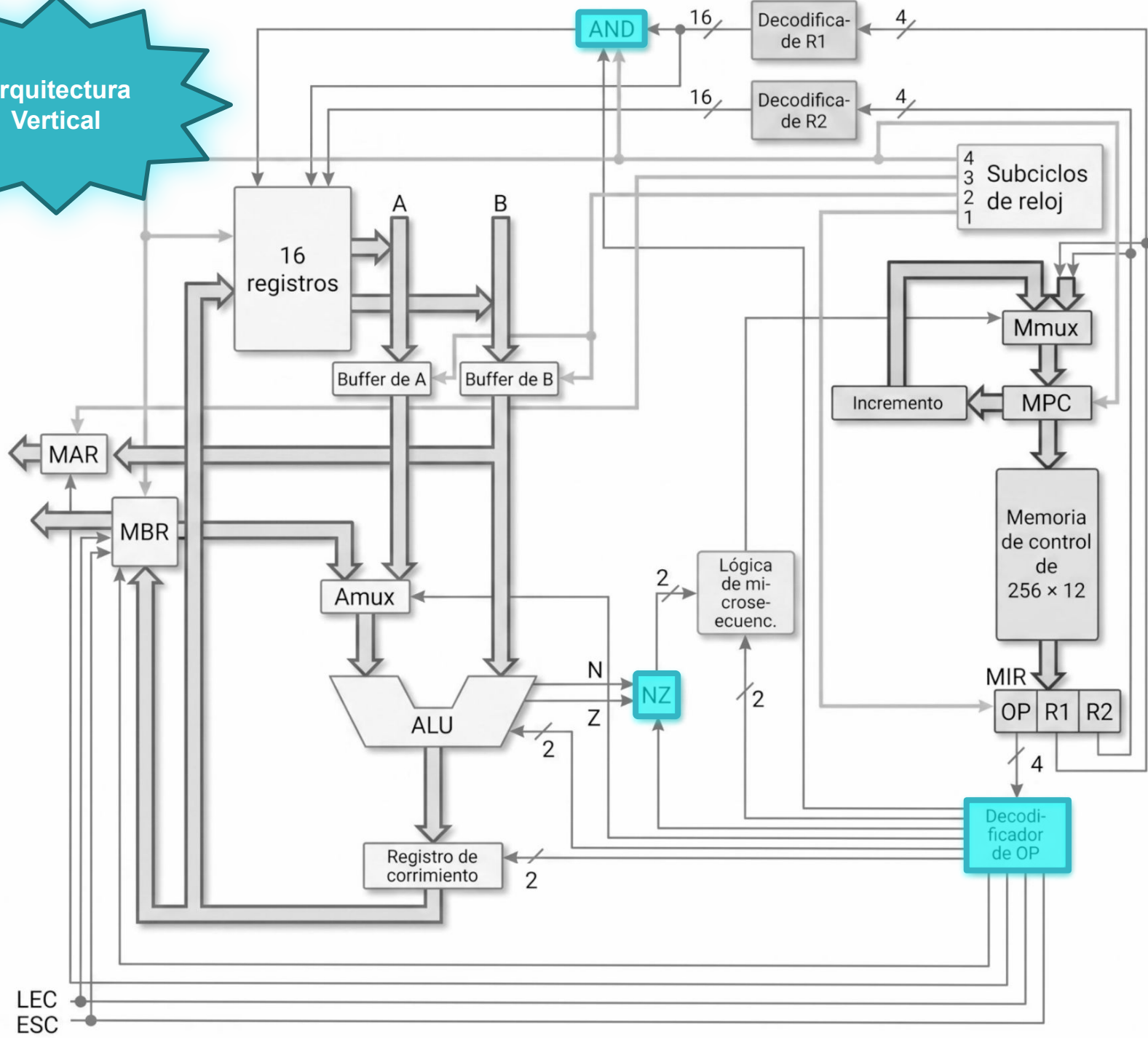




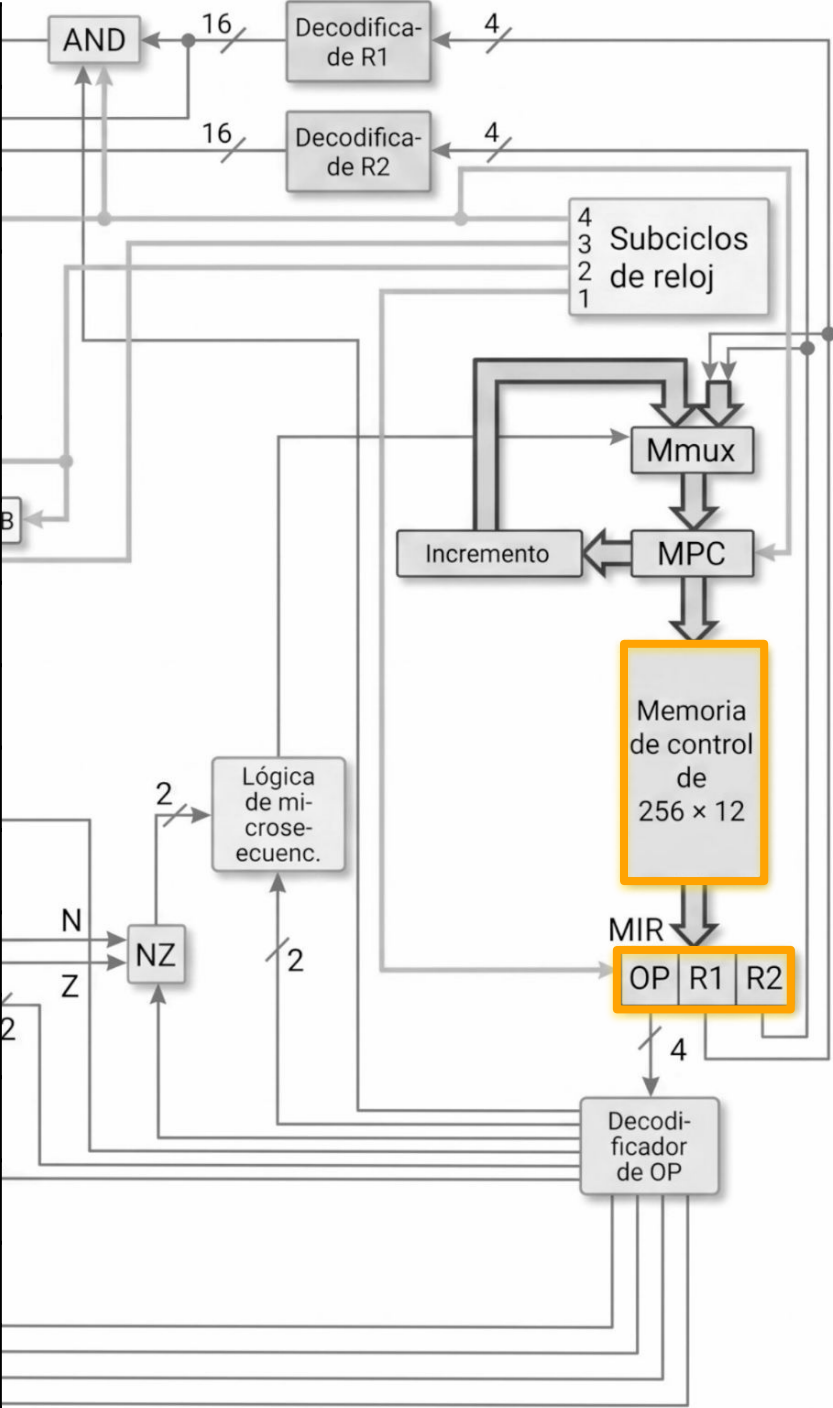




Arquitectura Vertical

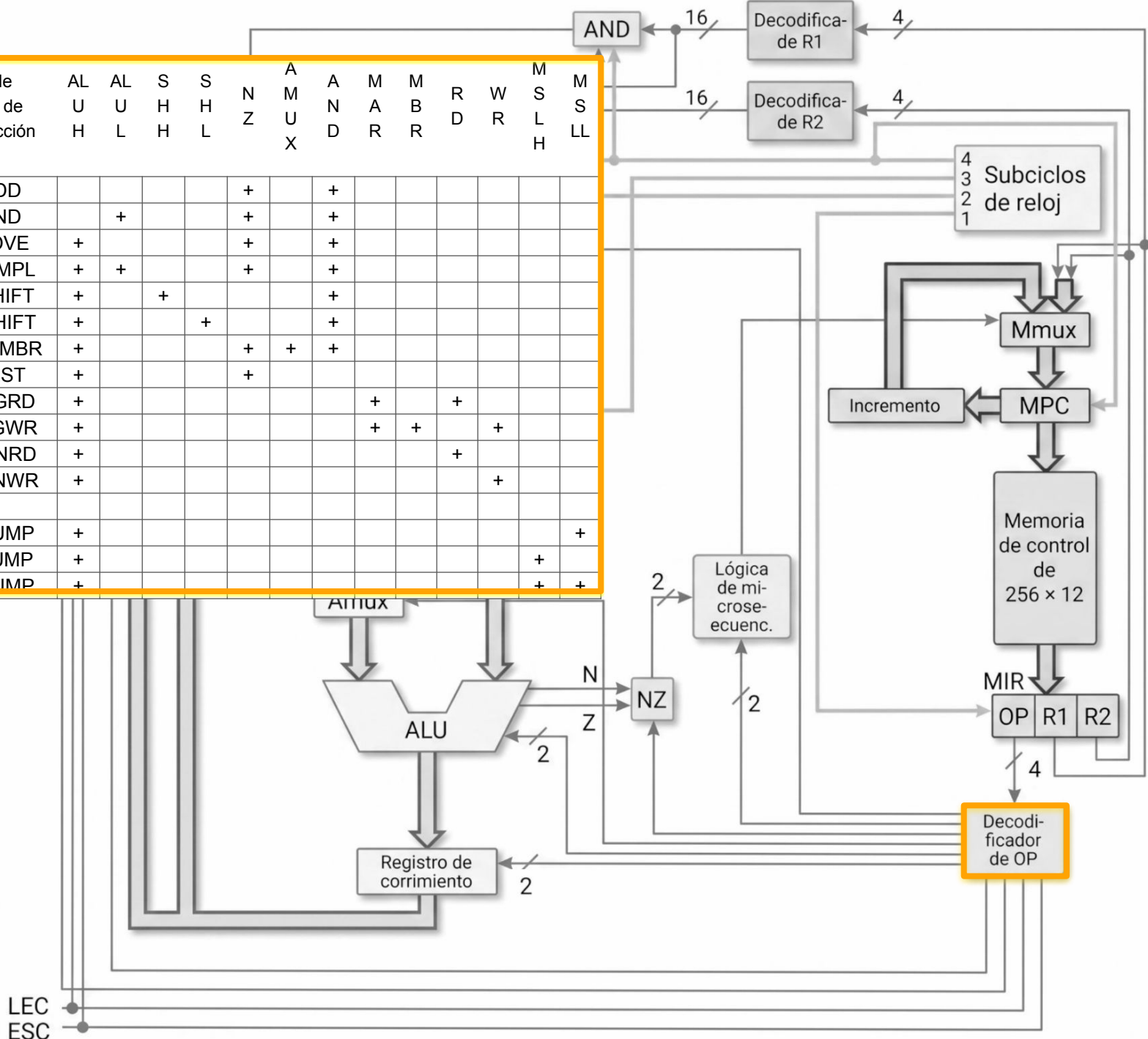


OP	Mnemónico	Instrucción
0000	ADD	Suma
0001	AND	Y bit a bit
0010	MOVE	Mueve registro
0011	COMPL	Complemento
0100	LSHIFT	Desplaza a la Izquierda
0101	RSHIFT	Desplaza a la Derecha
0110	GETMBR	Almacena MBR en registro
0111	TEST	Examina registro
1000	BEGRD	Comienza la Lectura
1001	BEGWR	Comienza la Escritura
1010	CONRD	Continúa la Lectura
1011	CONWR	Continúa la Escritura
1100		
1101	NJUMP	Salta si N=true
1110	ZJUMP	Salta si Z=true
1111	UJUMP	Salta siempre

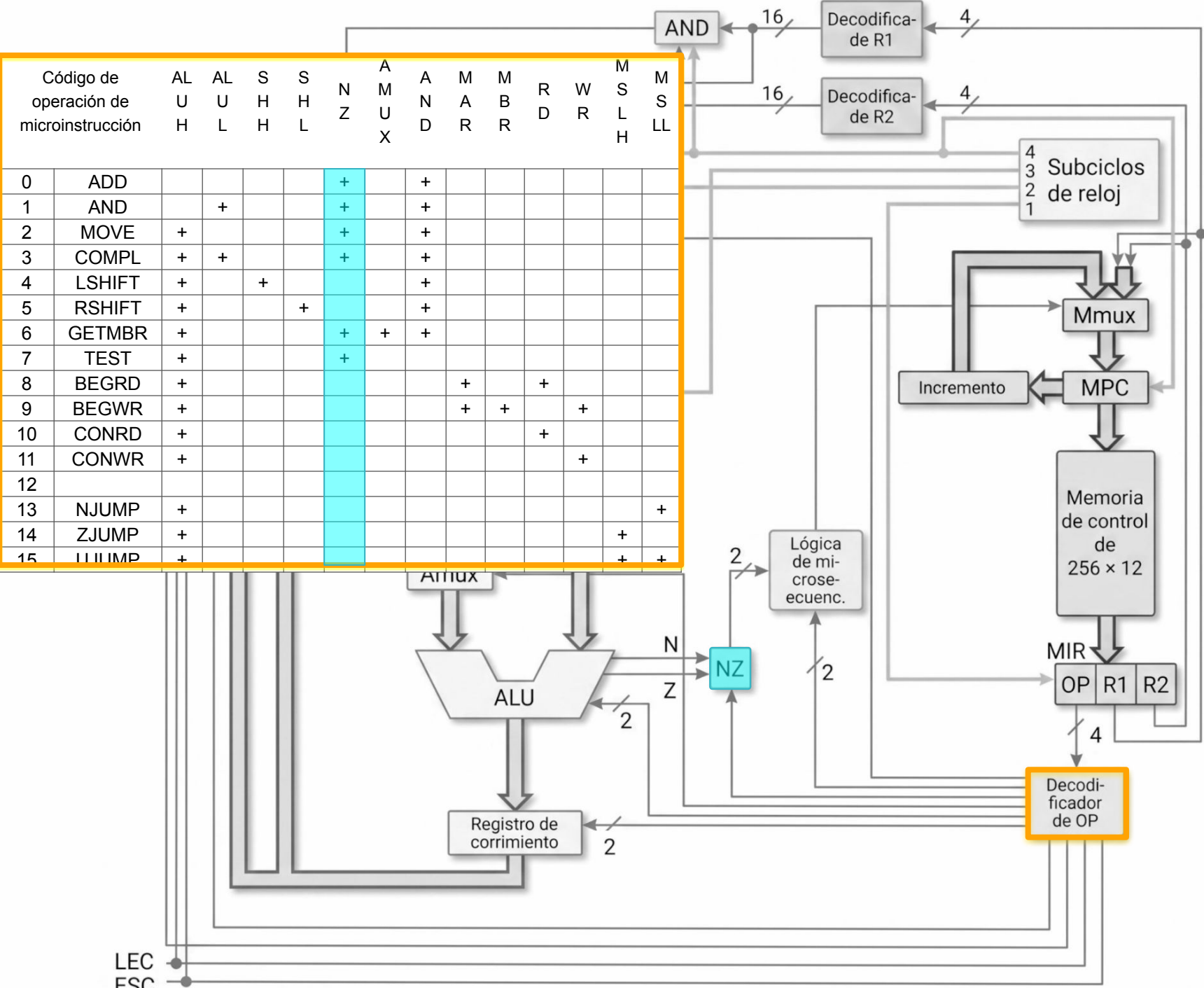


Binario	Mnemónico	Instrucción	Significado
0000	ADD	Suma	r1:=r1+r2
0001	AND	Y bit a bit	r1:=r1&r2
0010	MOVE	Mueve registro	r1:=r2
0011	COMPL	Complemento	r1:=inv(r2)
0100	LSHIFT	Desplaza a la Izquierda	r1:=r2<<1
0101	RSHIFT	Desplaza a la Derecha	r1:=r2>>1
0110	GETMBR	Almacena MBR en registro	r1:=MBR
0111	TEST	Examina registro	N:=(r2<0); Z:=(r2==0)
1000	BEGRD	Comienza la Lectura	MAR:=r1; rd
1001	BEGWR	Comienza la Escritura	MAR:=r1; MBR:=r2; wr
1010	CONRD	Continúa la Lectura	rd
1011	CONWR	Continúa la Escritura	wr
1100			
1101	NJUMP	Salta si N=true	if N then goto r1<<4 r2
1110	ZJUMP	Salta si Z=true	if Z then goto r1<<4 r2
1111	UJUMP	Salta siempre	goto r1<<4 r2

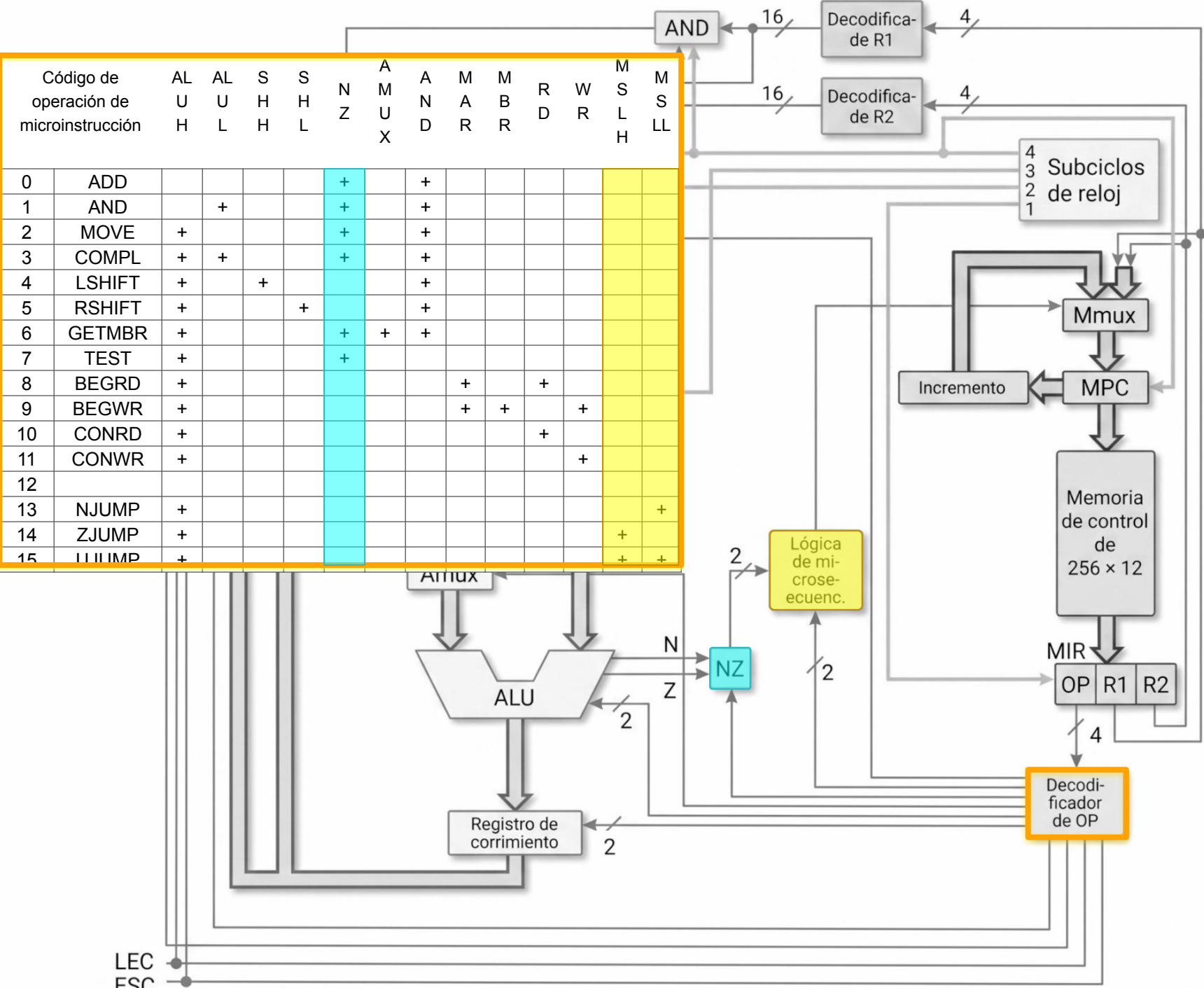
Código de operación de microinstrucción	AL U H	AL U L	S H H	S H L	N Z	A M U X	A N D	M A R	M B R	R D	W R	M S L H	M S L L
0	ADD				+		+						
1	AND		+		+		+						
2	MOVE	+			+		+						
3	COMPL	+	+		+		+						
4	LSHIFT	+		+			+						
5	RSHIFT	+			+		+						
6	GETMBR	+			+	+	+						
7	TEST	+			+								
8	BEGRD	+						+		+			
9	BEGWR	+						+	+		+		
10	CONRD	+								+			
11	CONWR	+									+		
12													
13	NJUMP	+											+
14	ZJUMP	+										+	+
15	IJUMP	+										+	+



Código de operación de microinstrucción	ALU	ALU	SH	SH	NZ	AMUX	AND	MAR	MBR	RD	WR	MSLH	MSLL
0	ADD				+		+						
1	AND		+		+		+						
2	MOVE	+			+		+						
3	COMPL	+	+		+		+						
4	LSHIFT	+		+			+						
5	RSHIFT	+			+		+						
6	GETMBR	+			+	+	+						
7	TEST	+			+								
8	BEGRD	+						+		+			
9	BEGWR	+						+	+		+		
10	CONRD	+								+			
11	CONWR	+									+		
12													
13	NJUMP	+											+
14	ZJUMP	+										+	
15	IJUMP	+										+	+



Código de operación de microinstrucción	AL U H	AL U L	S H H	S H L	N Z	A M U X	A N D	M A R	M B R	R D	W R	M S L H	M S L L
0	ADD				+		+						
1	AND		+		+		+						
2	MOVE	+			+		+						
3	COMPL	+	+		+		+						
4	LSHIFT	+		+			+						
5	RSHIFT	+			+		+						
6	GETMBR	+			+	+	+						
7	TEST	+			+								
8	BEGRD	+						+		+			
9	BEGWR	+						+	+		+		
10	CONRD	+								+			
11	CONWR	+									+		
12													
13	NJUMP	+											+
14	ZJUMP	+										+	
15	IJUMP	+										+	+



Binario	Mnemónico	Instrucción	Significado
0000	ADD	Suma	r1:=r1+r2
0001	AND	Y bit a bit	r1:=r1&r2
0010	MOVE	Mueve registro	r1:=r2
0011	COMPL	Complemento	r1:=inv(r2)
0100	Código de operación de microinstrucción	A L	A L
0101		U H	U L
0110	0	ADD	
0111	1	AND	
1000	2	MOVE	
1001	3	COMPL	
1010	4	LSHIFT	
1011	5	RSHIFT	
1100	6	GETMBR	
1101	7	TEST	
1110	8	BEGRD	
1111	9	BEGWR	
	10	CONRD	
	11	CONWR	
	12		
	13	NJUMP	
	14	ZJUMP	
	15	UJUMP	
		UJUMP	Salta siempre

Código de operación de microinstrucción		A L	A L	SH H	SH L	NZ	A M UX	AN D	M AR	M BR	R D	W R	M S H	M S L
		U H	U L											
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+				+						
5	RSHIFT	+			+			+						
6	GETMBR	+					+	+						
7	TEST	+				+								
8	BEGRD	+							+		+			
9	BEGWR	+							+	+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+

Conservar N y Z

Binario	Mnemónico	Instrucción	Significado
0000	ADD	Suma	r1:=r1+r2
0001	AND	Y bit a bit	r1:=r1&r2
0010	MOVE	Mueve registro	r1:=r2
0011	COMPL	Complemento	r1:=inv(r2)
0100	Código de operación de microinstrucción	A L	A L
0101		U H	U L
0110	0	ADD	
0111		AND	
1000	3	COMPL	
1001	4	LSHIFT	
1010	6	GETMBR	
1011	8	BEGRD	
1100	10	CONRD	
1101	12		
1110	14	ZJUMP	
1111	15	UJUMP	

Binario	Mnemónico	Instrucción	Significado													
0000	ADD	Suma	r1:=r1+r2													
0001	AND	Y bit a bit	r1:=r1&r2													
0010	MOVE	Código de operación de microinstrucción A L U H A L U L SH SH NZ A M UX AN M M R W S M														
0011	COMPL															
0100	LSHIFT															
0101	RSHIFT															
0110	GETMBR	0	ADD					+		+						
0111	TEST	1	AND		+			+		+						
1000	BEGRD	2	MOVE	+				+		+						
1001	BEGWR	3	COMPL	+	+			+		+						
1010	CONRD	4	LSHIFT	+		+				+						
1011	CONWR	5	RSHIFT	+			+			+						
1100		6	GETMBR	+				+	+	+						
		7	TEST	+				+								
		8	BEGRD	+							+		+			
		9	BEGWR	+							+		+		+	
		10	CONRD										+			
		11	CONWR											+		
		12														
		13	NJUMP	+											+	
		14	ZJUMP	+											+	
		15	UJUMP	+											+	+
1101	NJUMP	Salta si N=true		if N then goto r1<<4 r2												
1110	ZJUMP	Salta si Z=true		if Z then goto r1<<4 r2												
1111	UJUMP	Salta siempre		goto r1<<4 r2												

Binario	Mnemónico	Instrucción	Significado
0000	ADD	Suma	$r1 := r1 + r2$
0001	AND	Y bit a bit	$r1 := r1 \& r2$
0010	MOVE	Código de operación de microinstrucción	A L SH SH A AN M M R W S M U U H L M D AR BR D R L S H L UX H L
0011	COMPL		
0100	LSHIFT	0 ADD	Afecta la lógica de micro secuenciamiento Según el NZ anterior 
0101	RSHIFT	1 AND	
0110	GETMBR	2 MOVE	
0111	TEST	3 COMPL	
1000	BEGRD	4 LSHIFT	
1001	BEGWR	5 RSHIFT	
1010	CONRD	6 GETMBR	
1011	CONWR	7 TEST	
1100		8 BEGRD	
1101	NJUMP	9 BEGWR	
1110	ZJUMP	10 CONRD	
1111	UJUMP	11 CONWR	
		12	
		13 NJUMP	
		14 ZJUMP	
		15 UJUMP	
1101	NJUMP	Salta si N=true	if N then goto $r1 \ll 4 \mid r2$
1110	ZJUMP	Salta si Z=true	if Z then goto $r1 \ll 4 \mid r2$
1111	UJUMP	Salta siempre	goto $r1 \ll 4 \mid r2$

¿Cómo se implementa?

Código de operación de microinstrucción	A L U H	A L U L	SH H	SH L	NZ	A M UX	AN D	M AR	M BR	R D	W R	M S L H	M S L L
0	ADD				+		+						
1	AND		+		+		+						
2	MOVE	+			+		+						
3	COMPL	+	+		+		+						
4	LSHIFT	+		+			+						
5	RSHIFT	+			+		+						
6	GETMBR	+			+	+	+						
7	TEST	+			+								
8	BEGRD	+						+		+			
9	BEGWR	+						+	+		+		
10	CONRD	+								+			
11	CONWR	+									+		
12													
13	NJUMP	+											+
14	ZJUMP	+										+	
15	UJUMP	+										+	+

LEC
ESC

Registro de
corrimiento

2

Modifica-
R1

4

Modifica-
R2

4

4
3
2
1 Subciclos
de reloj

Mmux

Incremento

MPC

Memoria
de control
de
256 × 12

MIR

OP R1 R2

4

Decodi-
ficador
de OP

Implementación del decodificador de OP

Código de operación de microinstrucción	A L U H	A L U L	SH H	SH L	NZ	A M UX	AN D	M AR	M BR	R D	W R	M S L H	M S L L
0	ADD				+		+						
1	AND		+		+		+						
2	MOVE	+			+		+						
3	COMPL	+	+		+		+						
4	LSHIFT	+		+			+						
5	RSHIFT	+			+		+						
6	GETMBR	+			+	+	+						
7	TEST	+			+								
8	ECBP	+						+		+			
9	B							+	+		+		
10	CO									+			
11	CO										+		
12													
13	NJU												+
14	ZJU											+	
15	UJUMP	+										+	+

0100

Registro de
corrimiento

2

Modifica-
R1

4

Modifica-
R2

4

4
3
2
1 Subciclos
de reloj

Mmux

Incremento

MPC

Memoria
de control
de
256 × 12

MIR

OP R1 R2

4

Decodi-
ficador
de OP

LEC
ESC

Implementación del decodificador de OP

Código de operación de microinstrucción	A L U H	A L U L	SH H	SH L	NZ	A M UX	AN D	M AR	M BR	R D	W R	M S L H	M S L L
0	ADD				+		+						
1	AND		+		+		+						
2	MOVE	+			+		+						
3	COMPL	+	+		+		+						
4	LSHIFT	+		+			+						
5	RSHIFT	+		+			+						
6	GETMBR	+			+	+	+						
7	TEST				+								
8	BEG							+		+			
9	BEGWR							+	+		+		
10	CONRD									+			
11	CONWR										+		
12													
13	NJUMP												+
14	ZJUMP	+										+	
15	UJUMP	+										+	+

0101

Registro de
corrimiento

2

Modifica-
R1

4

Modifica-
R2

4

4
3
2
1 Subciclos
de reloj

Mmux

Incremento

MPC

Memoria
de control
de
256 × 12

MIR

OP R1 R2

4

Decodi-
ficador
de OP

LEC
ESC

Implementación del decodificador de OP

Código de operación de microinstrucción	A L U H	A L U L	SH H	SH L	NZ	A M UX	AN D	M AR	M BR	R D	W R	M S L H	M S L L
0	ADD				+		+						
1	AND		+		+		+						
2	MOVE	+			+		+						
3	COMPL	+	+		+		+						
4	LSHIFT	+		+			+						
5	RSHIFT	+		+			+						
6	GETMBR	+			+	+	+						
7	TEST				+								
8	BEG							+		+			
9	BEGWR							+	+		+		
10	CONRD									+			
11	CONWR										+		
12													
13	NJUMP												+
14	ZJUMP	+										+	
15	UJUMP	+										+	+

0101

Registro de
corrimiento

2

Modifica-
R1

4

Modifica-
R2

4

4
3
2
1 Subciclos
de reloj

Mmux

Incremento

MPC

Memoria
de control
de
256 × 12

MIR

OP R1 R2

4

Decodi-
ficador
de OP

LEC
ESC

Implementación del decodificador de OP

Código de operación de microinstrucción	A L U H	A L U L	SH H	SH L	NZ	A M UX	AN D	M AR	M BR	R D	W R	M S L H	M S L L
0	ADD				+		+						
1	AND		+		+		+						
2	MOVE	+			+		+						
3	COMPL	+	+		+		+						
4	LSHIFT	+		+			+						
5	RSHIFT	+			+		+						
6					+	+	+						
7					+								
8								+		+			
9								+	+		+		
10										+			
11											+		
12													
13	NJUMP	+											+
14	ZJUMP	+										+	
15	UJUMP	+										+	+

0001
0011
00?1

Registro de
corrimiento

2

Modifica-
R1

4

Modifica-
R2

4

4
3
2
1 Subciclos
de reloj

Mmux

Incremento

MPC

Memoria
de control
de
256 × 12

MIR

OP R1 R2

4

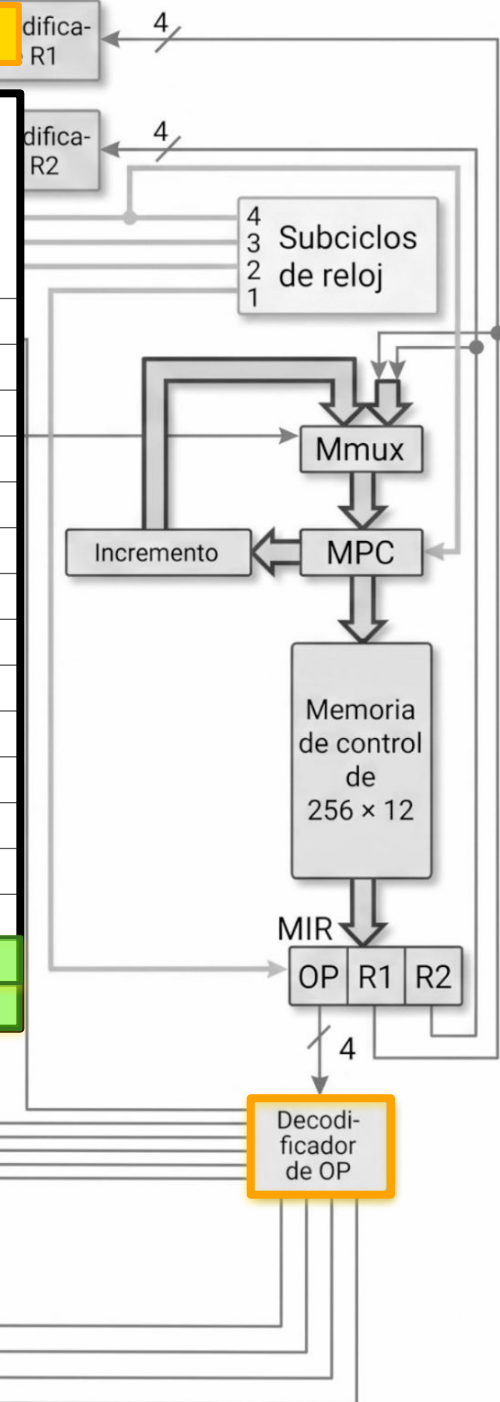
Decodi-
ficador
de OP

LEC
ESC

Implementación del decodificador de OP

Código de operación de microinstrucción	A L U H	A L U L	SH H	SH L	NZ	A M U X	AN D	M A R	M B R	R D	W R	M S L H	M S L L
0	ADD				+		+						
1	AND		+		+		+						
2	MOVE	+			+		+						
3	COMPL	+	+		+		+						
4	LSHIFT	+		+			+						
5	RSHIFT	+			+		+						
6					+	+	+						
7					+								
8								+		+			
9								+	+		+		
10										+			
11											+		
12													
13	NJUMP	+											+
14	ZJUMP	+										+	
15	UJUMP	+										+	+

1110
1111
111?



```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
   {STOD}
10: wr
   goto 0;
11: alu := tir;
   if n then goto 15;
12: mar := ir; rd; {ADDD}
13: rd;
14: a := mbr;
   ac := ac + a;
   goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
   a := inv(a);
18: ac := ac + a;
   goto 0;
19: tir := lshift(tir);
   if n then goto 25;
20: alu := tir;
   if n then goto 23;
21: alu := ac; {JPOS}
   if n then goto 0;
22: pc := ir;
   pc band(pc, amask);
   goto 0;
23: alu := ac; {IZER}
   if z then goto 22;
24: goto 0;
25: alu := tir;
   if n then goto 27;
26: pc:= ir; {JUMP}
   pc:= band(pc, amask);
   goto 0;
27: ac := ir; {LOCO}
   ac :=band(ac, amask);
   goto 0;
28: tir := lshift (tir):
   if n then goto 40;
29: tir:= lshift (tir);
   if n then goto 35;
30: alu := tir;
   if n then goto 33;
31: a := ir; {LODL}
   a := a + sp;
32: mar := a; rd;
   rd;
   ac := mbr;
   goto 0
33: a := ir; {STOL}
   a := a + sp;
34: mar := a; mbr := ac, wr;
   wr;
   goto 0;
35: alu := tir;
   if n then goto 38;
36: a := ir; {ADDL}
   a := a + sp;
37: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;
38: a := ir; {SUBL}
   a := a + sp;
39: mar := a; rd;
   rd;
   goto 99;
40: tir := lshift (tir );
   if n then goto 46;
41: alu:= tir;
   if n then goto 44;
42: alu:= ac; {JNEG}
   if n then goto 22;
43: goto 0;
44: alu := ac; {JNZE}
   if z then goto 0;
45: pc := ir;
   pc := band (pc, amask);
   goto 0;
46: tir := lshift(tir);
   if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
   wr;
49: pc := ir;
   pc := band(pc, amask);
   goto 0;
50: tir := lshift (tir );
   if n then goto 65;
51: tir := lshift(tir);
   if n then goto 59;
52: alu := tir;
   if n then goto 56;
53: mar := ac; rd; {PSHI}
   rd;
54: sp := sp + (-1);
55: a := mbr;
   mar := sp; mbr:=a, wr;
   wr;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;
58: a := mbr;
   mar := ac; mbr := a; wr;
   wr;
   goto 0;
59: alu := tir;
   if n then goto 62;
60: sp := sp + (-1); {PUSH}
61: mar:= sp; mbr :=ac; wr;
   wr;
   goto 0;
62: mar := sp; rd; {POP}
63: rd;
   sp := sp + 1;
64: ac := mbr;
   goto 0;
65: tir := lshift (tir ):
   if n then goto 73;
66: alu := tir;
   if n then goto 70;
67: mar := sp; rd; {RETN}
68: rd;
   sp := sp + 1;
69: pc := mbr;
   goto 0;
70: a := ac; {SWAP}
71: ac := sp;
72: sp:= a;
   goto 0;
73: alu := tir;
   if n then goto 76;
74: a := ir; {INSP}
   a := band(a, smask);
75: sp := sp + a;
   goto 0;
76: a := ir; {DESP}
   a := band(a, smask);
77: a := inv(a);
78: a := a + 1;
   sp := sp + a;
   goto 0;

```

Microprograma

0: mar := pc; rd;	23: alu := ac; {IZER}	38: a := ir; {SUBL}	58: a := mbr;
1: rd;	if z then goto 22;	a := a + sp;	mar := ac; mbr := a; wr;
pc := pc + 1;	24: goto 0;	39: mar := a; rd;	wr;
2: ir := mbr;	25: alu := tir;	rd;	goto 0;
tir := lshift(ir);	if n then goto 27;	goto 99;	59: alu := tir;
if n then goto 28;	26: pc:= ir; {JUMP}	40: tir := lshift (tir);	if n then goto 62;
3: tir := lshift(tir);	pc:= band(pc, amask);	if n then goto 46;	60: sp := sp + (-1); {PUSH}
if n then goto 19;	goto 0;	41: alu:= tir;	61: mar:= sp; mbr :=ac; wr;
4: tir := lshift(tir);	27: ac := ir; {LOCO}	if n then goto 44;	wr;
if n then goto 11;	ac :=band(ac, amask);	42: alu:= ac; {JNEG}	goto 0;
5: alu := tir;	goto 0;	if n then goto 22;	62: mar := sp; rd; {POP}
if n then goto 09;	28: tir := lshift (tir):	43: goto 0;	63: rd;
6: mar := ir; rd; {LODD}	if n then goto 40;	44: alu := ac; {JNZE}	sp := sp + 1;
7: rd;	29: tir:= lshift (tir);	if z then goto 0;	64: ac := mbr;
8: ac := mbr;	if n then goto 35;	45: pc := ir;	goto 0;
goto 0;	30: alu := tir;	pc := band (pc, amask);	65: tir := lshift (tir):
9: mar := ir; mbr:= ac; wr;	if n then goto 33;	goto 0;	if n then goto 73;
{STOD}	31: a := ir; {LODL}	46: tir := lshift(tir);	66: alu := tir;
10: wr	a := a + sp;	if n then goto 50;	if n then goto 70;
goto 0;	32: mar := a; rd;	47: sp := sp + (-1); {CALL}	67: mar := sp; rd; {RETN}
11: alu := tir;	rd;	48: mar := sp; mbr:= pc; wr;	68: rd;
if n then goto 15;	ac := mbr;	wr;	sp := sp + 1;
12: mar := ir; rd; {ADD}	goto 0;	49: pc := ir;	69: pc := mbr;
13: rd;	33: a := ir; {STOL}	pc := band(pc, amask);	goto 0;
14: a := mbr;	a := a + sp;	goto 0;	70: a := ac; {SWAP}
ac := ac + a;	34: mar := a; mbr := ac, wr;	50: tir := lshift (tir);	71: ac := sp;
goto 0;	wr;	if n then goto 65;	72: sp:= a;
15: mar := ir; rd; {SUBD}	goto 0;	51: tir := lshift(tir);	goto 0;
16: rd;	35: alu := tir;	if n then goto 59;	73: alu := tir;
99: ac := ac + 1;	if n then goto 38;	52: alu := tir;	if n then goto 76;
17: a := mbr;	36: a := ir; {ADDL}	if n then goto 56;	74: a := ir; {INSP}
a := inv(a);	a := a + sp;	53: mar := ac; rd; {PSHI}	a := band(a, smask);
18: ac := ac + a;	37: mar := a; rd;	rd;	75: sp := sp + a;
goto 0;	rd;	54: sp := sp + (-1);	goto 0;
19: tir := lshift(tir);	a := mbr;	55: a := mbr;	76: a := ir; {DESP}
if n then goto 25;	ac := ac + a;	mar := sp; mbr:=a, wr;	a := band(a, smask);
20: alu := tir;	goto 0;	wr;	77: a := inv(a);
if n then goto 23;		goto 0;	78: a := a + 1;
21: alu := ac; {JPOS}		56: mar := sp; rd; {POPI}	sp := sp + a;
if n then goto 0;		57: rd;	goto 0;
22: pc := ir;		sp := sp + 1;	
pc band(pc, amask);			
goto 0;			

Microprograma vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
                        {STOD}
10: wr
    goto 0;

```

Microprograma horizontal

```

0: mar := pc ; rd ;
1: pc := pc + 1 ; rd ;
2: ir := mbr ; if n then goto 28 ;
3: tir := lshift(ir + ir) ; if n
  then goto 19 ;
4: tir := lshift(tir) ; if n then
  goto 11 ;
5: alu := tir ; if n then goto 9 ;
6: mar := ir ; rd ;
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;
10: wr ; goto 0 ;

```

Microprograma vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
                        {STOD}
10: wr
    goto 0;

```

Microprograma horizontal

```

0: mar := pc ; rd ;
1: pc := pc + 1 ; rd ;
2: ir := mbr ; if n then goto 28 ;
3: tir := lshift(ir + ir) ; if n
  then goto 19 ;
4: tir := lshift(tir) ; if n then
  goto 11 ;
5: alu := tir ; if n then goto 9 ;
6: mar := ir ; rd ;
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;
10: wr ; goto 0 ;

```

Microprograma vertical

0: mar := pc; rd;

1: rd;

pc := pc + 1;

2: ir := mbr;

tir := lshift(ir);

if n then goto 28;

3: tir := lshift(tir);

if n then goto 19;

4: tir := lshift(tir);

1000

BEGRD

Comienza la Lectura

5: alu := tir;

if n then goto 09;

6: mar := ir; rd; {LODD}

7: rd;

8: ac := mbr;

goto 0;

9: mar := ir; mbr := ac; wr;

{STOD}

10: wr

goto 0;

Microprograma horizontal

0: mar := pc ; rd ;

1: pc := pc + 1 ; rd ;

2: ir := mbr ; if n then goto 28 ;

3: tir := lshift(ir + ir) ; if n then goto 19 ;

4: tir := lshift(tir) ; if n then goto 11 ;

5: alu := tir ; if n then goto 9 ;

7: rd ;

8: ac := mbr ; goto 0 ;

9: mar := ir ; mbr := ac ; wr ;

10: wr ; goto 0 ;

MAR:=r1; rd

Microprograma vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr := ac; wr;
   {STOD}
10: wr
    goto 0;

```

Microprograma horizontal

```

0: mar := pc ; rd ;
1: pc := pc + 1 ; rd ;
2: ir := mbr ; if n then goto 28 ;
3: tir := lshift(ir + ir) ; if n
  then goto 19 ;
4: tir := lshift(tir) ; if n then
  goto 11 ;
5: alu := tir ; if n then goto 9 ;
6: mar := ir ; rd ;
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;
10: wr ; goto 0 ;

```

Microprograma vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);

```

1010

CONRD

Continúa la Lectura

```

5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;   {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr := ac; wr;
   {STOD}
10: wr
    goto 0;

```

Microprograma horizontal

```

0: mar := pc ; rd ;
1: pc := pc + 1 ; rd ;
2: ir := mbr ; if n then goto 28 ;
3: tir := lshift(ir + ir) ; if n
  then goto 19 ;
4: tir := lshift(tir) ; if n then
  goto 11 ;
5: alu := tir ; if n then goto 9 ;
6: alu := tir ;
7: alu := tir ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;
10: wr ; goto 0 ;

```

Microprograma vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);

```

1010

CONRD

Continúa la Lectura

0000

ADD

Suma

```

6: mar := ir; rd;   {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr := ac; wr;
   {STOD}
10: wr
    goto 0;

```

Microprograma horizontal

```

0: mar := pc ; rd ;
1: pc := pc + 1 ; rd ;
2: ir := mbr ; if n then goto 28 ;
3: tir := lshift(ir + ir) ; if n
  then goto 19 ;
4: tir := lshift(tir) ; if n then
  goto 11 ;
5: alu := tir ; if n then goto 9 ;

```

rd

r1:=r1+r2

```

9: mar := ir ; mbr := ac ; wr ;
10: wr ; goto 0 ;

```

Microprograma vertical

```
0: mar := pc; rd;
1: rd;
  pc := pc + 1;
2: ir := mbr;
  tir := lshift(ir);
  if n then goto 28;
3: tir := lshift(tir);
  if n then goto 19;
4: tir := lshift(tir);
  if n then goto 11;
5: alu := tir;
  if n then goto 09;
6: mar := ir; rd;    {LODD}
7: rd;
8: ac := mbr;
  goto 0;
9: mar := ir; mbr:= ac; wr;
  {STOD}
10: wr
  goto 0;
```

Microprograma horizontal

```
0: mar := pc ; rd ;
1: pc := pc + 1 ; rd ;
2: ir := mbr ; if n then goto 28 ;
3: tir := lshift(ir + ir) ; if n
  then goto 19 ;
4: tir := lshift(tir) ; if n then
  goto 11 ;
5: alu := tir ; if n then goto 9 ;
6: mar := ir ; rd ;
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;
10: wr ; goto 0 ;
```

Microprograma vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);

```

0110

GETMBR

Almacena MBR en registro

r1:=MBR

```

5: alu := tir;
   if n then goto 09;
6: mar := ir; rd;   {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
   {STOD}
10: wr
    goto 0;

```

Microprograma horizontal

```

0: mar := pc ; rd ;
1: pc := pc + 1 ; rd ;
2: ir := mbr ; if n then goto 28 ;
3: tir := lshift(ir + ir) ; if n
  then goto 19 ;
4: tir := lshift(tir) ; if n then
  goto 11 ;
5: alu := tir ; if n then goto 9 ;
6: alu := tir ;
7: alu := tir ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;
10: wr ; goto 0 ;

```

Microprograma vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);

```

0110

GETMBR

Almacena MBR en registro

r1:=MBR

0100

LSHIFT

Desplaza a la Izquierda

r1:=r2<<1

```

   if n then goto 05;
6: mar := ir; rd;   {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
   {STOD}
10: wr
    goto 0;

```

Microprograma horizontal

```

0: mar := pc ; rd ;
1: pc := pc + 1 ; rd ;
2: ir := mbr ; if n then goto 28 ;
3: tir := lshift(ir + ir) ; if n
  then goto 19 ;
4: tir := lshift(tir) ; if n then
  goto 11 ;
5: alu := tir ; if n then goto 9 ;

9: mar := ir ; mbr := ac ; wr ;
10: wr ; goto 0 ;

```

Microprograma vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);

```

0110	GETMBR	Almacena MBR en registro	r1:=MBR
0100	LSHIFT	Desplaza a la Izquierda	r1:=r2<<1
1101	NJUMP	Salta si N=true	if N then goto r1<<4 r2

```

7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
   {STOD}
10: wr
    goto 0;

```

Microprograma horizontal

```

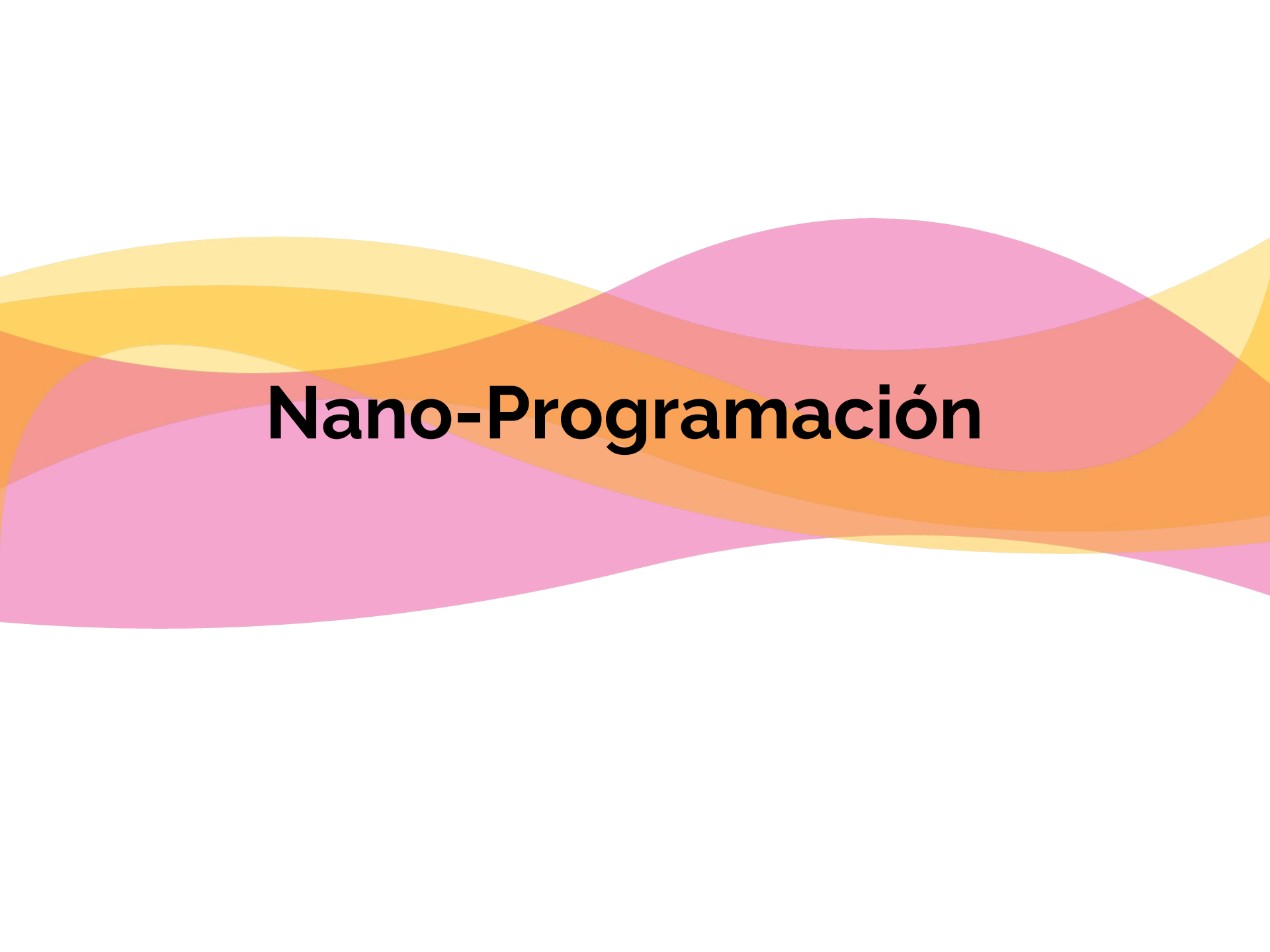
0: mar := pc ; rd ;
1: pc := pc + 1 ; rd ;
2: ir := mbr ; if n then goto 28 ;
3: tir := lshift(ir + ir) ; if n
  then goto 19 ;
4: tir := lshift(tir) ; if n then
  goto 11 ;
5: alu := tir ; if n then goto 9 ;

```

```

10: wr ; goto 0 ;

```



Nano-Programación

0: mar := pc; rd;	23: alu := ac; {IZER}	38: a := ir; {SUBL}	58: a := mbr;
1: rd;	if z then goto 22;	a := a + sp;	mar := ac; mbr := a; wr;
pc := pc + 1;	24: goto 0;	39: mar := a; rd;	wr;
2: ir := mbr;	25: alu := tir;	rd;	goto 0;
tir := lshift(tir);	if n then goto 27;	goto 99;	59: alu := tir;
if n then goto 28;	26: pc:= ir; {JUMP}	40: tir := lshift(tir);	if n then goto 62;
3: tir := lshift(tir);	pc:= band(pc, amask);	if n then goto 46;	60: sp := sp + (-1); {PUSH}
if n then goto 19;	goto 0;	41: alu:= tir;	61: mar:= sp; mbr:=ac; wr;
4: tir := lshift(tir);	27: ac := ir; {LOCO}	if n then goto 44;	wr;
if n then goto 11;	ac :=band(ac, amask);	42: alu:= ac; {JNEG}	goto 0;
5: alu := tir;	goto 0;	if n then goto 22;	62: mar := sp; rd; {POP}
if n then goto 09;	28: tir := lshift (tir);	43: goto 0;	63: rd;
6: mar := ir; rd; {LODD}	if n then goto 40;	44: alu := ac; {JNZE}	sp := sp + 1;
7: rd;	29: tir := lshift (tir);	if z then goto 0;	64: ac := mbr;
8: ac := mbr;	if n then goto 35;	45: pc := ir;	goto 0;
goto 0;	30: alu := tir;	pc := band (pc, amask);	65: tir := lshift (tir);
9: mar := ir; mbr:= ac; wr;	if n then goto 33;	goto 0;	if n then goto 73;
{STOD}	31: a := ir; {LODL}	46: tir := lshift(tir);	66: alu := tir;
10: wr	a := a + sp;	if n then goto 50;	if n then goto 70;
goto 0;	32: mar := a; rd;	47: sp := sp + (-1); {CALL}	67: mar := sp; rd; {RETN}
11: alu := tir;	rd;	48: mar := sp; mbr:= pc; wr;	68: rd;
if n then goto 15;	ac := mbr;	wr;	sp := sp + 1;
12: mar := ir; rd; {ADD}	goto 0;	49: pc := ir;	69: pc := mbr;
13: rd;	33: a := ir; {STOL}	pc := band(pc, amask);	goto 0;
14: a := mbr;	a := a + sp;	goto 0;	70: a := ac; {SWAP}
ac := ac + a;	34: mar := a; mbr := ac, wr;	50: tir := lshift (tir);	71: ac := sp;
goto 0;	wr;	if n then goto 65;	72: sp:= a;
15: mar := ir; rd; {SUBD}	goto 0;	51: tir := lshift(tir);	goto 0;
16: rd;	35: alu := tir;	if n then goto 59;	73: alu := tir;
99: ac := ac + 1;	if n then goto 38;	52: alu := tir;	if n then goto 76;
17: a := mbr;	36: a := ir; {ADDL}	if n then goto 56;	74: a := ir; {INSP}
a := inv(a);	a := a + sp;	53: mar := ac; rd; {PSHI}	a := band(a, smask);
18: ac := ac + a;	37: mar := a; rd;	rd;	75: sp := sp + a;
goto 0;	rd;	54: sp := sp + (-1);	goto 0;
19: tir := lshift(tir);	a := mbr;	55: a := mbr;	76: a := ir; {DESP}
if n then goto 25;	ac := ac + a;	mar := sp; mbr:=a, wr;	a := band(a, smask);
20: alu := tir;	goto 0;	wr;	77: a := inv(a);
if n then goto 23;		goto 0;	78: a := a + 1;
21: alu := ac; {JPOS}		56: mar := sp; rd; {POPI}	sp := sp + a;
if n then goto 0;		57: rd;	goto 0;
22: pc := ir;		sp := sp + 1;	
pc band(pc, amask);			
goto 0;			

Micro-Instrucciones repetidas

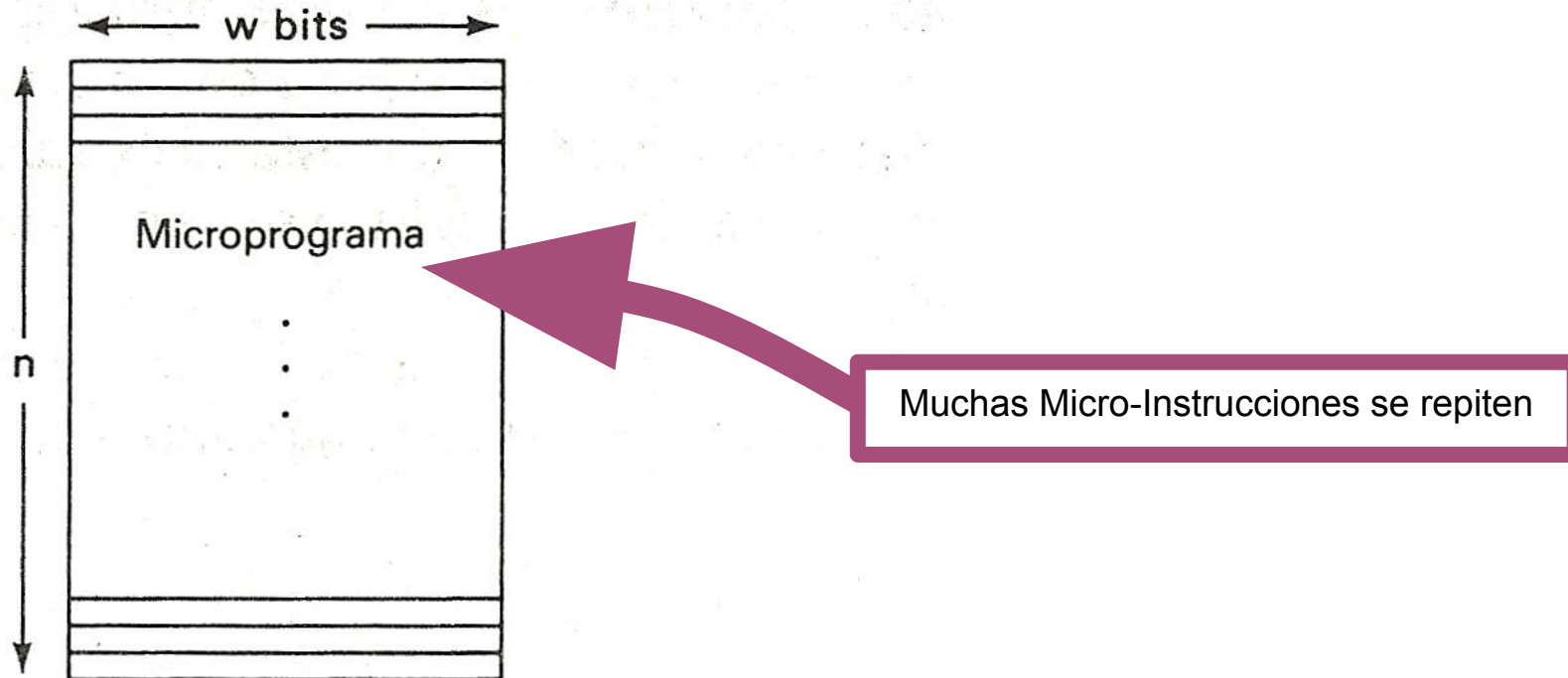
```
0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(tir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
   {STOD}
10: wr
   goto 0;
11: alu := tir;
   if n then goto 15;
12: mar := ir; rd; {ADD}
13: rd;
14: a := mbr;
   ac := ac + a;
   goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
   a := inv(a);
18: ac := ac + a;
   goto 0;
19: tir := lshift(tir);
   if n then goto 25;
20: alu := tir;
   if n then goto 23;
21: alu := ac; {JPOS}
   if n then goto 0;
22: pc := ir;
   pc band(pc, amask);
   goto 0;
```

```
if n then goto 27;
26: pc:= ir; {JUMP}
   pc:= band(pc, amask);
   goto 0;
27: ac := ir; {LOCO}
   ac :=band(ac, amask);
   goto 0;
28: tir := lshift (tir);
   if n then goto 40;
29: tir := lshift (tir);
   if n then goto 35;
30: alu := tir;
   if n then goto 33;
31: a := ir; {LODL}
   a := a + sp;
32: mar := a; rd;
   rd;
   ac := mbr;
   goto 0;
33: a := ir; {STOL}
   a := a + sp;
34: mar := a; mbr := ac, wr;
   wr;
   goto 0;
35: alu := tir;
   if n then goto 38;
36: a := ir; {ADDL}
   a := a + sp;
37: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;
```

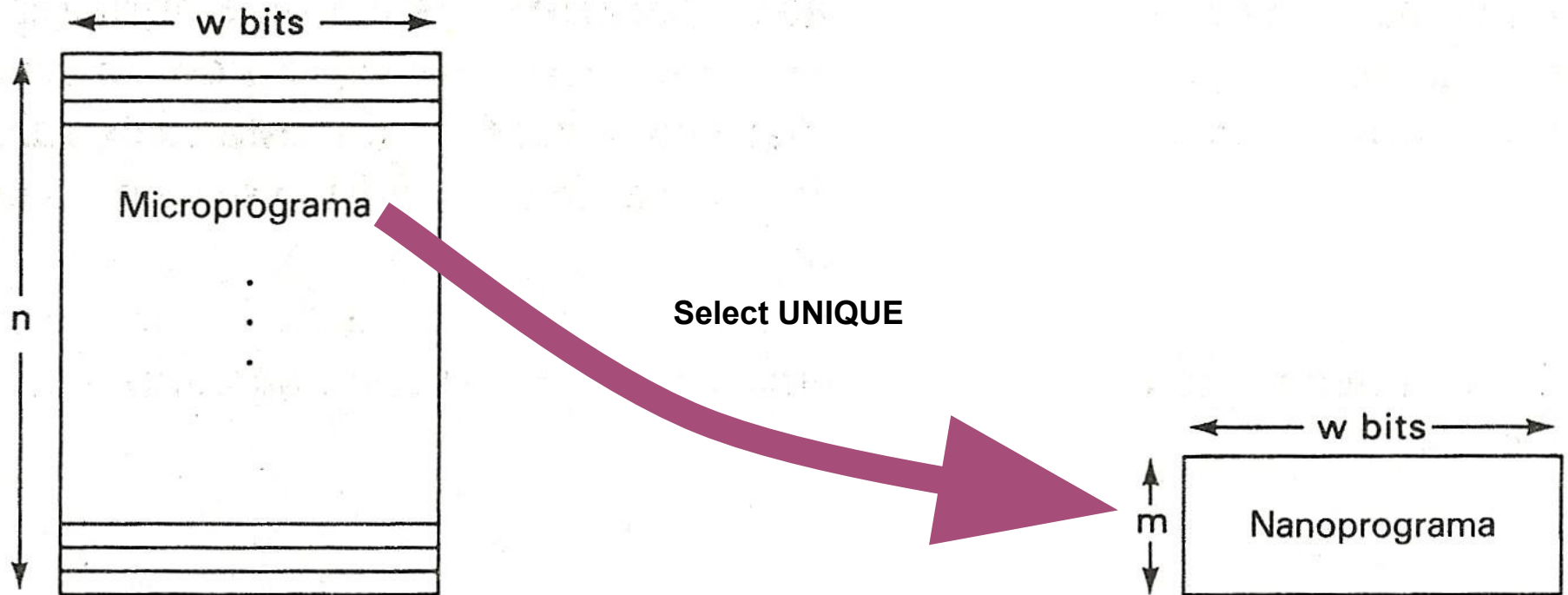
```
goto 99;
40: tir := lshift(tir);
   if n then goto 46;
41: alu:= tir;
   if n then goto 44;
42: alu:= ac; {JNEG}
   if n then goto 22;
43: goto 0;
44: alu := ac; {JNZE}
   if z then goto 0;
45: pc := ir;
   pc := band (pc, amask);
   goto 0;
46: tir := lshift(tir);
   if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
   wr;
49: pc := ir;
   pc := band(pc, amask);
   goto 0;
50: tir := lshift (tir );
   if n then goto 65;
51: tir := lshift(tir);
   if n then goto 59;
52: alu := tir;
   if n then goto 56;
53: mar := ac; rd; {PSHI}
   rd;
54: sp := sp + (-1);
55: a := mbr;
   mar := sp; mbr:=a, wr;
   wr;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;
```

```
8: a := mbr;
   mar := ac; mbr := a; wr;
   wr;
   goto 0;
59: alu := tir;
   if n then goto 62;
60: sp := sp + (-1); {PUSH}
61: mar:= sp; mbr :=ac; wr;
   wr;
   goto 0;
62: mar := sp; rd; {POP}
63: rd;
   sp := sp + 1;
64: ac := mbr;
   goto 0;
65: tir := lshift (tir );
   if n then goto 73;
66: alu := tir;
   if n then goto 70;
67: mar := sp; rd; {RETN}
68: rd;
   sp := sp + 1;
69: pc := mbr;
   goto 0;
70: a := ac; {SWAP}
71: ac := sp;
72: sp:= a;
   goto 0;
73: alu := tir;
   if n then goto 76;
74: a := ir; {INSP}
   a := band(a, smask);
75: sp := sp + a;
   goto 0;
76: a := ir; {DESP}
   a := band(a, smask);
77: a := inv(a);
78: a := a + 1;
   sp := sp + a;
   goto 0;
```

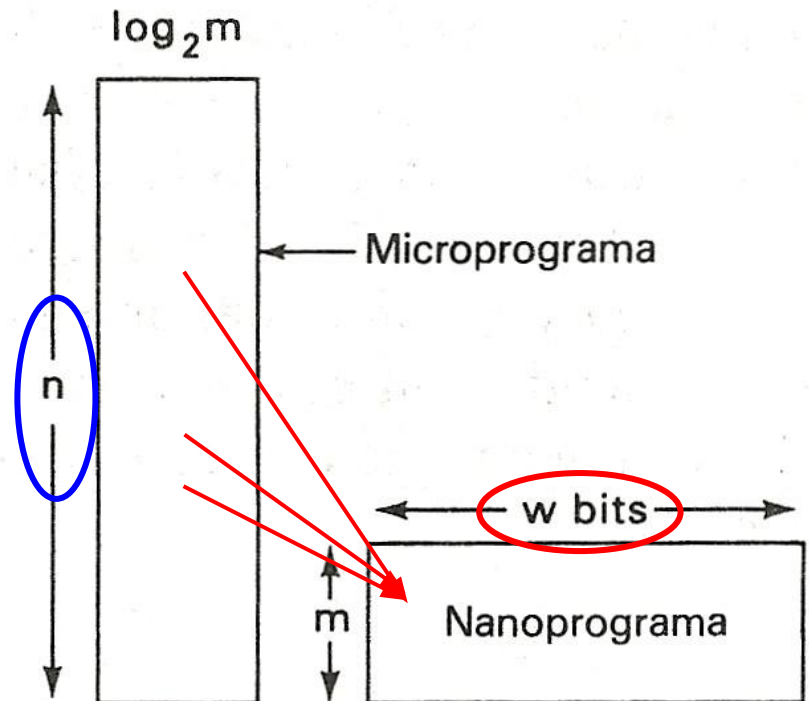
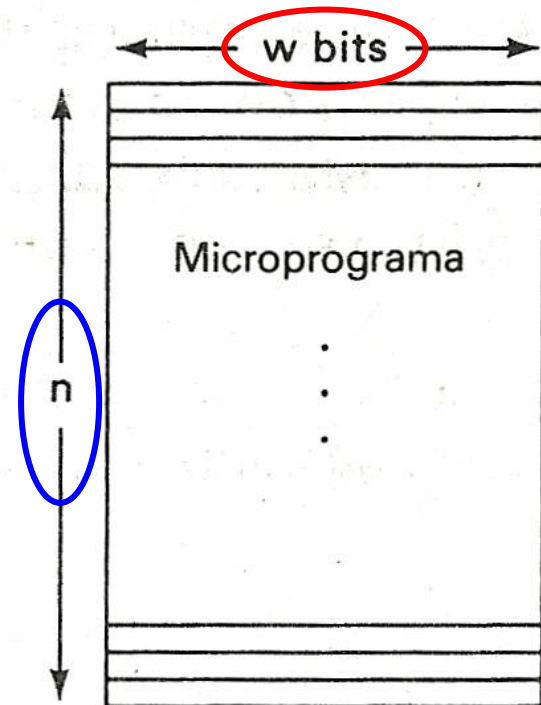
Nano programación



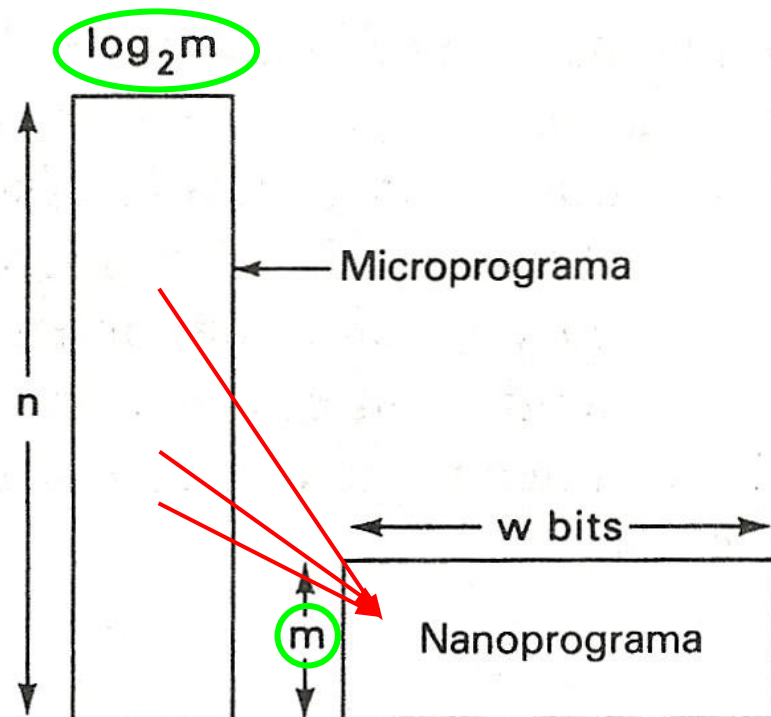
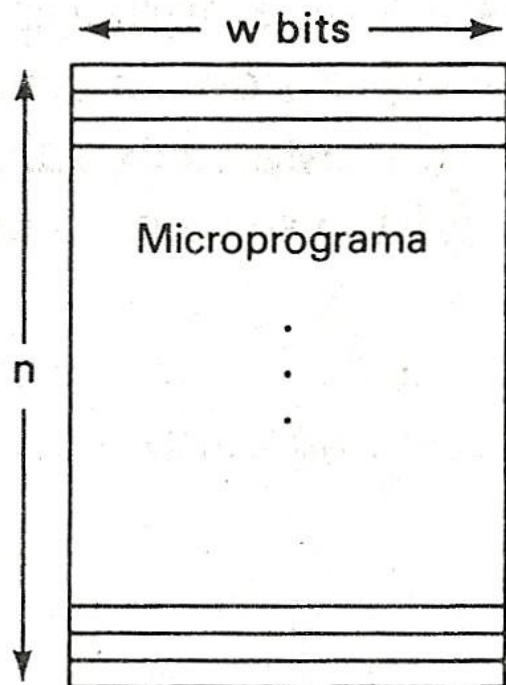
Nano programación

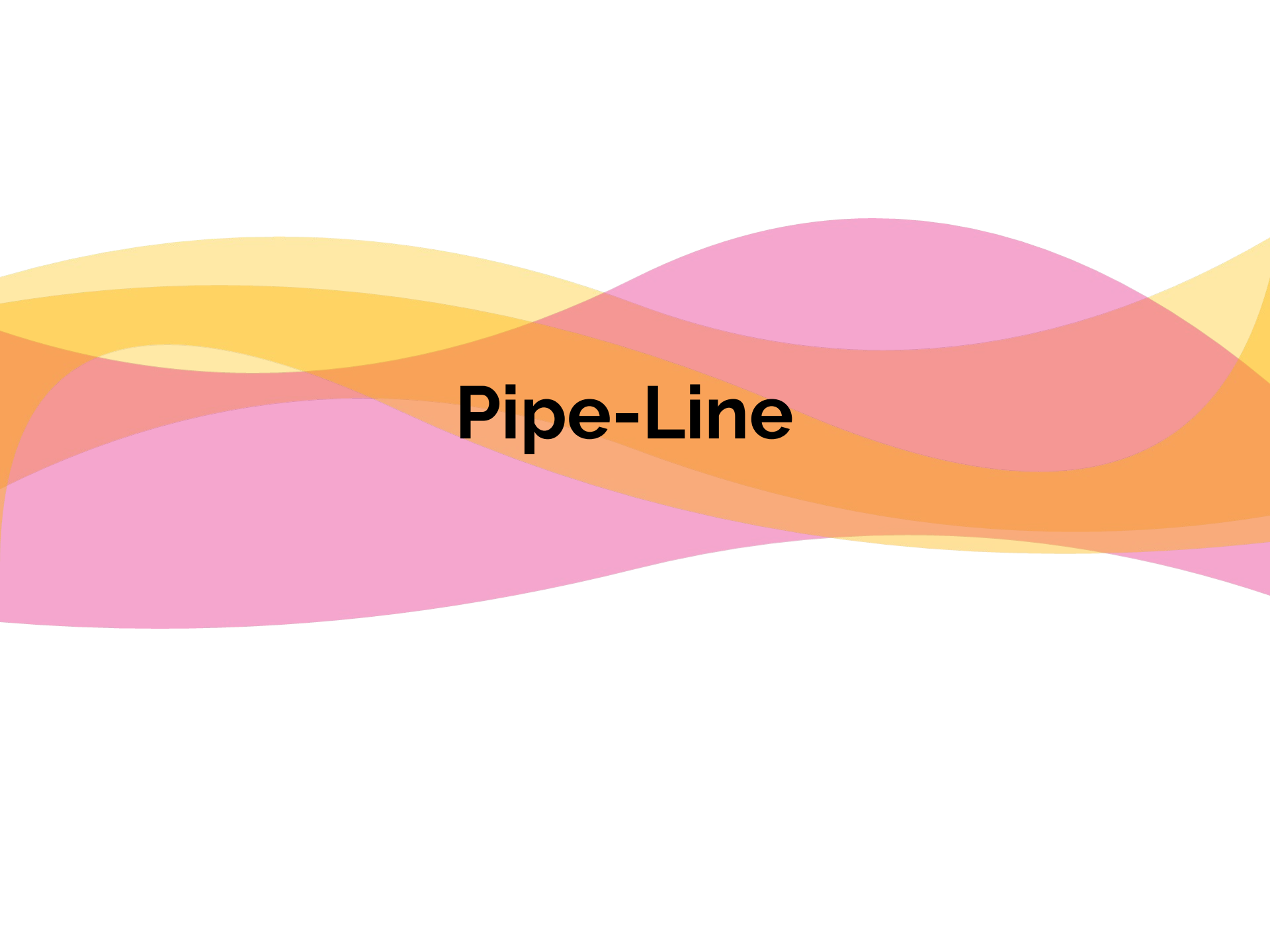


Nano memoria



Nano memoria





Pipe-Line

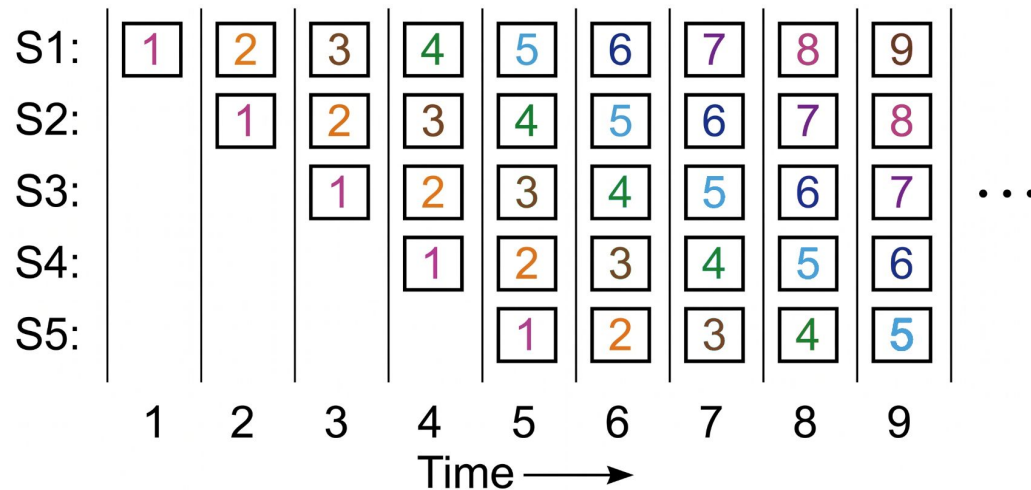
Pipe Line

- Mejora del rendimiento
- Varias unidades específicas

Pipe Line



- Mejora del rendimiento
- Varias unidades específicas



Pipe Line



- Mejora del rendimiento
- Varias unidades específicas
- 30% de las instrucciones son ***saltos***
 - ***incondicionales***
 - ***condicionales***
 - ***iteraciones***

Pipe Line



- Mejora del rendimiento
- Varias unidades específicas
- 30% de las instrucciones son **saltos**

	1	2	3	4	5	6	7	8	9	10	11		
Extracción de instrucciones	1	2	B					4	5	6	7	8	9
Decodificación de instrucciones		1	2	B					4	5	6	7	8
Cálculo de direcciones			1	2	B					4	5	6	7
Extracción de operandos				1	2	B					4	5	6
Ejecución					1	2	B					4	5

Pipe Line



Los compiladores optimizan el ejecutable a fin de minimizar la Penalización por Salto

	1	2	3	4	5	6	7	8	9	10	11		
Extracción de instrucciones	1	2	B					4	5	6	7	8	9
Decodificación de instrucciones		1	2	B					4	5	6	7	8
Cálculo de direcciones			1	2	B					4	5	6	7
Extracción de operandos				1	2	B					4	5	6
Ejecución					1	2	B					4	5

Arquitectura Vertical

Ejecución del microprograma



Fundamentos de Arquitectura de computadoras

Un cambio en el control de los datos. Ahora se codifican las microinstrucciones para utilizar menos memoria de control.

- Arquitectura Vertical
- Nanoprogramción
- Pipeline

